



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Design and Implementation of an Efficient Priority-Based Emergency Patient Management System Using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing

*Submitted in the partial fulfilment for the Course
of*

CSA0307- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

CSE(AI)

Submitted by

Sancia Sanky(192572005)

Under the Supervision of

Dr. K. Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering(AI)
Programme:	B.E
Course Code & Course Name	CSA0307 Data Structure
Academic Year / Batch	2 nd year (2025-2029)
Faculty Name	Dr.K.Kiruthika
Assignment Title	Design and Implementation of an Efficient Priority-Based Emergency Patient Management System Using Appropriate Data Structures for Severity-Based Treatment and Arrival-Order Processing
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	CO2: Apply the suitable Abstract Data type for construction of the smart city emergency system CO4: Make use of heaps, priority queue and hashing techniques for graph storage and route cache for repeated queries in C. CO5: Develop robust smart city emergency system by implementing shortest path algorithms such as Dijkstra's algorithm, Floyd-Warshall.
Bloom's Taxonomy Level	L4 – Analyze
SDG Mapping (SDG 1-17)	SDG-9 and SDG-11
Industry / Societal Relevance	SDG 9 - Industry, Innovation, and Infrastructure SDG 11 – Sustainable Cities and Communities;

B. Assignment Problem / Challenge

The challenge is to design an efficient Emergency Patient Management System that prioritizes patients based on the severity of their condition. Critically ill patients must be treated first, while patients with the same severity are treated in arrival order (FIFO). The system should support patient registration, priority assignment, searching, updating, and removal using an appropriate data structure such as a Priority Queue.

C. Problem Statement

A hospital emergency department receives patients continuously throughout the day, and each patient may require a different level of medical attention depending on the severity of their condition. For every patient, the system needs to maintain important information such as a unique patient ID, arrival time, severity level, and assigned doctor. Patients with critical or life-threatening conditions must be given higher priority and treated before patients with less severe conditions. However, when two or more patients have the same severity level, they should generally be treated in the order in which they arrived at the emergency department, following the First-In-First-Out (FIFO) principle within the same priority level. As the number of patients may increase rapidly, the hospital requires an efficient data-structure-based solution for registering new patients, assigning priorities, selecting the next patient for treatment, updating patient information, and removing patients after treatment. Students has to design and implement an appropriate data structure for managing the emergency patient queue by evaluating the suitability of a simple queue, array, linked list, priority queue, or another suitable data structure. Justify the selection of the most appropriate data structure based on the requirements of priority-based treatment and arrival-order processing. Clearly describe the major operations involved, including patient insertion, priority assignment, identifying the next patient for treatment, deletion of treated patients, searching for a particular patient, and updating patient or doctor information. Analyze the time and space complexity of these operations in the best-case, average-case, and worst-case scenarios wherever applicable. In addition, critically discuss the limitations and possible problems that could occur if a conventional FIFO queue were used instead of a priority-based data structure. Explain how using a simple FIFO queue could cause critically ill patients to wait behind patients with less severe conditions, potentially resulting in delayed treatment and serious consequences. Finally, explain how the selected data structure can improve the efficiency, fairness, responsiveness, and reliability of patient management in a busy hospital emergency department.

D. Requirements and Constraints

- **Functional:** Find the shortest route between two intersections.
- **Performance:** Route calculation within **200 ms**.
- **Technical:** 50 nodes, 100 weighted edges; use **Dijkstra/A***.
- **Safety:** Avoid blocked or unavailable roads.
- **Reliability:** Provide accurate routes and handle traffic updates.
- **User:** Simple interface to select source and destination.
- **Security:** Protect vehicle/location data.
- **Cost:** Use low-cost/free software and tools.

E. Student Work

1. Problem Understanding and Formulation

1.1 What is the problem?

A hospital emergency department receives patients continuously, and each patient may have a different medical condition and severity level. Patients with critical or life-threatening conditions must be treated before patients with less severe conditions.

The system must maintain important information about every patient, including:

- Patient ID
- Arrival time
- Severity level
- Assigned doctor

The main challenge is to manage patients according to **priority**, while also maintaining **First-In-First-Out (FIFO)** order among patients having the same severity.

For example, if three patients have severity levels 5, 2 and 4, the patient with severity 5 should be treated first. If two patients have the same severity level, the patient who arrived earlier should be treated first.

Therefore, a conventional FIFO queue is not sufficient. A **Priority Queue** is more appropriate because it can select the most critical patient efficiently while maintaining arrival order within the same priority.

1.2 Expected Outcomes

The expected outcomes of the system are:

1. Register new emergency patients efficiently.
2. Assign priority according to severity.
3. Select the highest-priority patient for treatment.
4. Maintain FIFO order among patients with equal severity.
5. Remove patients after treatment.
6. Search for a particular patient.
7. Update patient or doctor information.
8. Display the current waiting patients.
9. Analyze the time and space complexity of operations.
10. Improve the efficiency and responsiveness of emergency patient management.

1.3 Available Information / Data

Data	Description
Patient ID	Unique identification number
Arrival Time	Time at which patient enters emergency department
Severity Level	Indicates seriousness of patient's condition
Assigned Doctor	Doctor responsible for treatment
Priority	Priority calculated from severity
Treatment Status	Waiting / Under Treatment / Completed

A sample severity classification can be:

Severity	Condition	Priority
5	Critical / Life-threatening	Highest
4	Very Serious	High
3	Serious	Medium
2	Moderate	Low
1	Minor	Lowest

For equal severity, **earlier arrival time gets higher treatment priority.**

1.4 Assumptions

1. Every patient has a unique Patient ID.
2. Severity is represented using levels from 1 to 5.
3. A higher severity value represents a more critical condition.
4. Patients with higher severity are treated first.
5. Patients with the same severity follow FIFO order.
6. Arrival time is recorded when the patient is registered.
7. Every patient can have an assigned doctor.
8. A patient is removed from the waiting structure after treatment.
9. The system is intended for emergency department queue management.
10. The system assumes that the severity level assigned by medical staff is valid.

1.5 Constraints

- Patient IDs must be unique.
- The system should efficiently handle a rapidly increasing number of patients.
- Critical patients must not unnecessarily wait behind less severe patients.
- Equal-severity patients should maintain arrival order.
- Patient information should be accurate.
- Searching and updating should be supported.
- The data structure should use reasonable memory.

- The system should provide reliable and responsive patient management.

2. Application of Course Knowledge

2.1 Abstract Data Type

The system can be represented using a **Priority Queue Abstract Data Type**.

The main operations are:

- insert()
- peek()
- delete()
- search()
- update()
- display()

The priority queue ensures that the patient with the highest severity is selected first.

2.2 Selected Data Structure

The most suitable data structure for this problem is a **Priority Queue implemented using a Binary Heap**.

Each heap element stores:

```
Patient {  
    patientID  
    arrivalTime  
    severity  
    doctor  
}
```

The priority comparison is:

Higher Severity → Higher Priority

If Severity is Equal:

Earlier Arrival Time → Higher Priority

Thus, the priority can be represented conceptually as:

Priority = (Severity, Arrival Order)

For implementation, a **Max-Heap** can be used where:

1. Higher severity has higher priority.
2. If severity is equal, smaller arrival sequence number has higher priority.

2.3 Why Priority Queue is Selected?

Different data structures were considered.

<i>DataStructure</i>	<i>Advantage</i>	<i>Limitation</i>	<i>Suitability</i>
Simple Queue	Easy FIFO processing	Cannot handle severity priority	Low
Array	Simple storage	Searching and shifting may be inefficient	Medium
Linked List	Dynamic memory	Finding highest priority can take $O(n)$	Medium
Priority Queue	Efficient priority-based processing	More complex than simple queue	High
Heap	Efficient insertion and deletion of highest priority	Searching arbitrary patient is $O(n)$	High

A simple queue cannot satisfy the main requirement because it always processes patients according to arrival time.

A priority queue is selected because it directly supports **priority-based treatment** and can maintain FIFO order among patients having equal severity.

2.4 Priority Queue Processing

Suppose the following patients arrive:

Patient	Arrival Order	Severity
P101	1	3
P102	2	5
P103	3	4
P104	4	5
P105	5	2

The treatment order will be:

P102 → P104 → P103 → P101 → P105

P102 and P104 both have severity 5, so P102 is treated first because it arrived earlier.

This satisfies both requirements:

Severity Priority + FIFO for Equal Severity

2.5 Major Operations

Patient Insertion

When a new patient arrives:

1. Generate/record a unique Patient ID.

2. Record arrival time.
3. Record severity.
4. Record assigned doctor.
5. Insert the patient into the priority queue.
6. Restore heap property.

Identify Next Patient

The patient at the root of the heap has the highest priority.

Therefore:

peek() → returns highest-priority patient

Delete Treated Patient

After treatment:

1. Remove the root patient.
2. Move the last heap element to the root.
3. Perform heapify-down.
4. Continue with the next highest-priority patient.

Search Patient

Search the heap for the required Patient ID.

Since a heap is optimized for priority rather than arbitrary searching, this operation may require checking multiple elements.

Update Patient

Patient information such as doctor or severity can be updated.

If severity changes, the patient's position in the priority queue may also need to be adjusted.

2.6 Mathematical Model

The priority comparison can be represented as:

$$P_i > P_j$$

if:

$$Severity_i > Severity_j$$

or if:

$$Severity_i = Severity_j$$

and

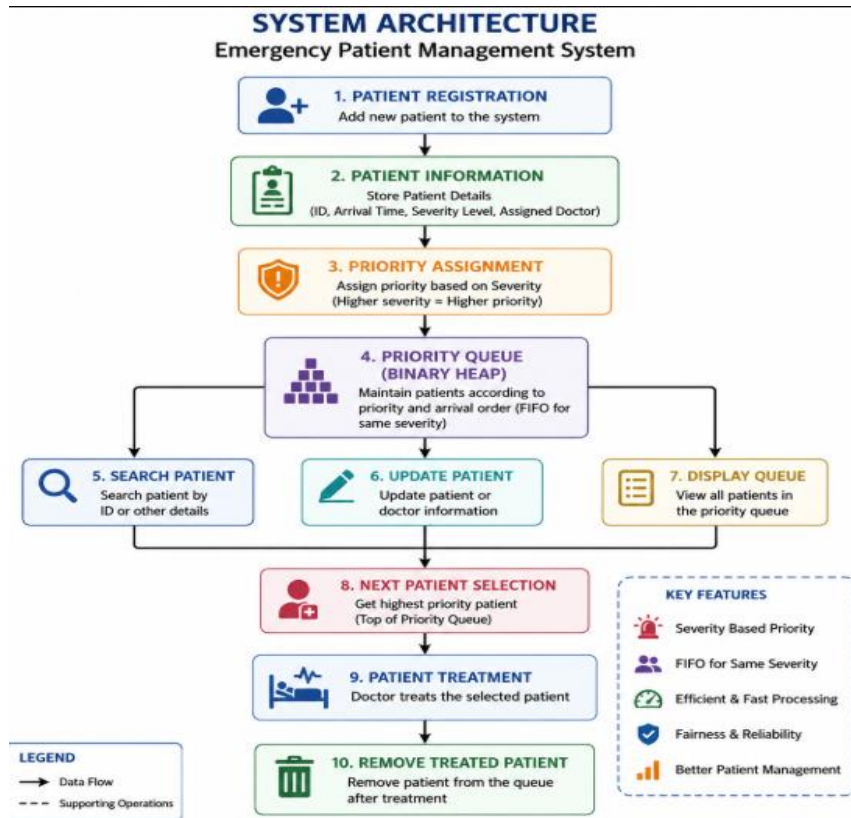
$$ArrivalOrder_i < ArrivalOrder_j$$

Therefore, a patient with higher severity always gets higher priority. For equal severity, the smaller arrival order gets higher priority.

3. Solution / Design / Methodology

3.1 System Architecture

The proposed system consists of the following modules:



3.2 System Modules

Module 1 – Patient Registration

Accepts:

- Patient ID
- Arrival time
- Severity
- Doctor name

Module 2 – Priority Assignment

Assigns treatment priority based on severity.

Module 3 – Priority Queue

Stores patients according to:

Severity → Arrival Order

Module 4 – Next Patient Selection

Selects the highest-priority patient.

Module 5 – Patient Treatment

The selected patient is marked as under treatment.

Module 6 – Patient Removal

After treatment, the patient is removed from the priority queue.

Module 7 – Search

Searches for a patient using Patient ID.

Module 8 – Update

Allows modification of:

- Severity
- Doctor
- Patient information

Module 9 – Display

Displays all waiting patients and their priority.

3.3 Algorithm

Algorithm: Insert Patient

START

Read Patient ID

Read Arrival Order

Read Severity

Read Doctor

Create Patient Record

Insert Patient into Priority Queue

Compare patient with parent node

If patient has higher priority:

 Swap with parent

Repeat until heap property is satisfied

STOP

Algorithm: Select Next Patient

START

If Priority Queue is empty:

 Display "No patients waiting"

Else:

 Select root patient

 Display patient information

STOP

Algorithm: Remove Treated Patient

START

If queue is empty:

 Display "No patient available"

Else:

 Remove root patient

 Move last patient to root

 Perform heapify-down

 Display next patient

STOP

Algorithm: Search Patient

START

Read Patient ID

For every patient in Priority Queue:

 If Patient ID matches:

 Display patient details

 STOP

Display "Patient not found"

STOP

Algorithm: Update Patient

START

Read Patient ID

Search for patient

If patient is found:

 Update required information

 If severity is changed:

 Restore heap property

Else:

 Display "Patient not found"

STOP

3.4 Pseudocode

The assignment deliverables specifically require **pseudocode, implementation and results, and GitHub upload.**

CLASS Patient

 patientID

 arrivalOrder

 severity

 doctor

FUNCTION higherPriority(p1, p2)

 IF p1.severity > p2.severity

 RETURN TRUE

 ELSE IF p1.severity == p2.severity

 IF p1.arrivalOrder < p2.arrivalOrder

 RETURN TRUE

 RETURN FALSE

FUNCTION insert(patient)

 Add patient to heap

 Perform heap-up

FUNCTION getNextPatient()

IF heap is empty

RETURN NULL

RETURN root of heap

FUNCTION removePatient()

IF heap is empty

RETURN NULL

patient = root

Replace root with last element

Remove last element

Perform heap-down

RETURN patient

FUNCTION search(patientID)

FOR each patient in heap

IF patient.patientID == patientID

RETURN patient

RETURN NULL

FUNCTION update(patientID, newSeverity, newDoctor)

patient = search(patientID)

IF patient exists

Update severity

Update doctor

Restore heap property

FUNCTION display()

FOR each patient

Display patient information

END

3.5 Example Implementation in C

```
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Patient {
    int id;
    int arrival;
    int severity;
    char doctor[30];
};

struct Patient heap[MAX];
int size = 0;

int higherPriority(struct Patient a, struct Patient b) {
    if (a.severity != b.severity)
        return a.severity > b.severity;

    return a.arrival < b.arrival;
}

void swap(struct Patient *a, struct Patient *b) {
    struct Patient temp = *a;
    *a = *b;
    *b = temp;
}

void insert(struct Patient p) {
    int i = size++;
    heap[i] = p;

    while (i > 0) {
        int parent = (i - 1) / 2;

        if (!higherPriority(heap[i], heap[parent]))
            break;
    }
}
```

```
        swap(&heap[i], &heap[parent]);
        i = parent;
    }
}
```

```
void heapifyDown(int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < size && higherPriority(heap[left], heap[largest]))
        largest = left;

    if (right < size && higherPriority(heap[right], heap[largest]))
        largest = right;

    if (largest != i) {
        swap(&heap[i], &heap[largest]);
        heapifyDown(largest);
    }
}
```

```
void treatPatient() {
    if (size == 0) {
        printf("No patients waiting.\n");
        return;
    }

    printf("\nTreating Patient ID: %d\n", heap[0].id);
    printf("Severity: %d\n", heap[0].severity);
    printf("Doctor: %s\n", heap[0].doctor);

    heap[0] = heap[--size];

    if (size > 0)
        heapifyDown(0);
}
```



```
}
```

```
void searchPatient(int id) {  
    for (int i = 0; i < size; i++) {  
        if (heap[i].id == id) {  
            printf("\nPatient Found\n");  
            printf("ID: %d\n", heap[i].id);  
            printf("Severity: %d\n", heap[i].severity);  
            printf("Doctor: %s\n", heap[i].doctor);  
            return;  
        }  
    }  
}
```

```
    printf("Patient not found.\n");  
}
```

```
void display() {  
    if (size == 0) {  
        printf("No patients waiting.\n");  
        return;  
    }  
}
```

```
    printf("\nWaiting Patients:\n");
```

```
    for (int i = 0; i < size; i++) {  
        printf("ID: %d | Arrival: %d | Severity: %d | Doctor: %s\n",  
            heap[i].id,  
            heap[i].arrival,  
            heap[i].severity,  
            heap[i].doctor);  
    }  
}
```

```
int main() {  
    int choice, id;  
  
    while (1) {
```

```

printf("\n--- Emergency Patient Management ---\n");
printf("1. Add Patient\n");
printf("2. Treat Next Patient\n");
printf("3. Search Patient\n");
printf("4. Display Patients\n");
printf("5. Exit\n");
printf("Enter choice: ");
scanf("%d", &choice);

if (choice == 1) {
    struct Patient p;

    printf("Enter Patient ID: ");
    scanf("%d", &p.id);

    p.arrival = size + 1;

    printf("Enter Severity (1-5): ");
    scanf("%d", &p.severity);

    printf("Enter Doctor: ");
    scanf("%s", p.doctor);

    insert(p);

    printf("Patient added successfully.\n");
}
else if (choice == 2) {
    treatPatient();
}
else if (choice == 3) {
    printf("Enter Patient ID: ");
    scanf("%d", &id);
    searchPatient(id);
}
else if (choice == 4) {
    display();
}

```

```

    }
    else if (choice == 5) {
        break;
    }
    else {
        printf("Invalid choice.\n");
    }
}

return 0;
}

```

4. Use of Modern Tools

The following tools can be used to develop and test the project.

Tool	Purpose
C	Main programming language
GCC	Compilation of C program
VS Code	Source code development
Code::Blocks	C programming and testing
Git	Version control
GitHub	Project source-code submission
Draw.io	System architecture/flow diagrams

The sample assignment also expects evidence of implementation and tool usage as part of the submission.

5. Results and Validation

5.1 Sample Input

Consider the following patients:

Patient ID	Arrival Order	Severity	Doctor
101	1	3	Kumar
102	2	5	Ravi
103	3	2	Anitha
104	4	5	Priya
105	5	4	Kumar

Expected Treatment Order

102 → 104 → 105 → 101 → 103

Reason

- Patient 102 has severity 5 and arrived before patient 104.
- Patient 104 also has severity 5, so it comes after patient 102.
- Patient 105 has severity 4.
- Patient 101 has severity 3.
- Patient 103 has severity 2.

Therefore, the priority queue correctly combines **severity priority with FIFO ordering**.

5.2 Sample Output

--- Emergency Patient Management ---

Patient added successfully.

Patient added successfully.

Patient added successfully.

Patient added successfully.

Patient added successfully.

Waiting Patients:

ID: 102 | Arrival: 2 | Severity: 5 | Doctor: Ravi

ID: 104 | Arrival: 4 | Severity: 5 | Doctor: Priya

ID: 105 | Arrival: 5 | Severity: 4 | Doctor: Kumar

ID: 101 | Arrival: 1 | Severity: 3 | Doctor: Kumar

ID: 103 | Arrival: 3 | Severity: 2 | Doctor: Anitha

Treating Patient ID: 102

Severity: 5

Doctor: Ravi

Treating Patient ID: 104

Severity: 5

Doctor: Priya

Emergency Patient Management System

Priority-based treatment using severity and arrival order

23:18:34

Total Patients

3

Critical Patients

0

Waiting

3

Treated

0

Register Patient

Patient ID

Example: P101

Patient Name

Enter patient name

Severity

3 - Moderate

Assigned Doctor

Example: Dr. Kumar

Add Patient to Queue

Priority rule: Higher severity is treated first. If severity is equal, the patient who arrived earlier is treated first (FIFO).

Emergency Queue

Treat Next Patient

Clear Queue

Priority	ID	Name	Severity	Arrival	Doctor
#1	P102	Sam	4 - Serious	2	Dr.Naveen
#2	P101	Dan	3 - Moderate	1	Dr.Mona
#3	P103	Dhruvi	2 - Mild	3	Dr.Avinash

Source Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Emergency Patient Management System</title>
<style>
*{box-sizing:border-box;margin:0;padding:0}
body{font-family:Arial,sans-serif;background:#f4f7fb;color:#172033}
header{background:linear-gradient(135deg,#0b4f6c,#087f8c);color:white;padding:24px 6%;display:flex;justify-content:space-between;align-items:center;gap:20px}
header h1{font-size:28px} header p{margin-top:6px;opacity:.9}
.container{width:88%;max-width:1200px;margin:25px auto}
.stats{display:grid;grid-template-columns:repeat(4,1fr);gap:16px;margin-bottom:22px}
.card{background:white;border-radius:14px;padding:20px;box-shadow:0 5px 18px #00000010}
.stat h2{font-size:28px;margin-top:8px}
.layout{display:grid;grid-template-columns:340px 1fr;gap:22px}
.form-card h2,.queue-card h2{margin-bottom:18px}
label{display:block;font-weight:bold;margin:13px 0 6px 0}
input,select{width:100%;padding:11px;border:1px solid #d5dce5;border-radius:8px;font-size:15px}
button{border:0;border-radius:8px;padding:11px 16px;cursor:pointer;font-weight:bold}
.primary{background:#087f8c;color:white;width:100%;margin-top:18px}
.primary:hover{background:#066b76}
.actions{display:flex;gap:10px;margin-bottom:16px;flex-wrap:wrap}
.actions button{background:#e8eef3}
.treat{background:#198754!important;color:white}
.clear{background:#dc3545!important;color:white}
.table-wrap{overflow:auto}
table{width:100%;border-collapse:collapse;min-width:650px}
th,td{text-align:left;padding:12px;border-bottom:1px solid #e7ebef}
th{background:#f1f5f8}
.badge{padding:5px 9px;border-radius:20px;font-size:12px;font-weight:bold}
.s5{background:#ffd6d6;color:#a40000}.s4{background:#ffe6cc;color:#9a4d00}
.s3{background:#ffffbf;color:#735f00}.s2{background:#d9f3ff;color:#00627a}.s1{background:#dfff5d;color:#176b17}
.empty{text-align:center;padding:40px;color:#7a8490}
.note{margin-top:18px;background:#eef8fa;padding:14px;border-left:4px solid #087f8c;border-radius:8px;font-size:14px}
@media(max-width:850px){.layout{grid-template-columns:1fr}.stats{grid-template-columns:repeat(2,1fr)}header{display:block}}
@media(max-width:500px){.stats{grid-template-columns:1fr}header h1{font-size:22px}}
</style>
</head>
<body>
```

```

<header>
<div>
  <h1> 🏥 Emergency Patient Management System</h1>
  <p>Priority-based treatment using severity and arrival order</p>
</div>
<strong id="clock"></strong>
</header>

<div class="container">
  <section class="stats">
    <div class="card stat"><span>Total Patients</span><h2 id="total">0</h2></div>
    <div class="card stat"><span>Critical Patients</span><h2 id="critical">0</h2></div>
    <div class="card stat"><span>Waiting</span><h2 id="waiting">0</h2></div>
    <div class="card stat"><span>Treated</span><h2 id="treated">0</h2></div>
  </section>

  <section class="layout">
    <div class="card form-card">
      <h2> 📄 Register Patient</h2>
      <form id="patientForm">
        <label>Patient ID</label>
        <input id="patientId" type="text" placeholder="Example: P101" required>

        <label>Patient Name</label>
        <input id="patientName" type="text" placeholder="Enter patient name" required>

        <label>Severity</label>
        <select id="severity" required>
          <option value="5">5 - Critical</option>
          <option value="4">4 - Serious</option>
          <option value="3" selected>3 - Moderate</option>
          <option value="2">2 - Mild</option>
          <option value="1">1 - Normal</option>
        </select>

        <label>Assigned Doctor</label>
        <input id="doctor" type="text" placeholder="Example: Dr. Kumar" required>

        <button class="primary" type="submit">Add Patient to Queue</button>
      </form>
    </div>
  </section>
</div>

```

```

        <label>Assigned Doctor</label>
        <input id="doctor" type="text" placeholder="Example: Dr. Kumar" required>

        <button class="primary" type="submit">Add Patient to Queue</button>
      </form>

      <div class="note">
        <b>Priority rule:</b> Higher severity is treated first. If severity is equal, the patient who arrived earlier is treated first (FIFO).
      </div>
    </div>

    <div class="card queue-card">
      <h2> 🏥 Emergency Queue</h2>
      <div class="actions">
        <button class="treat" onclick="treatNext()">▶ Treat Next Patient</button>
        <button class="clear" onclick="clearQueue()">Clear Queue</button>
      </div>
      <div class="table-wrap">
        <table>
          <thead>
            <tr><th>Priority</th><th>ID</th><th>Name</th><th>Severity</th><th>Arrival</th><th>Doctor</th></tr>
          </thead>
          <tbody id="queueBody"></tbody>
        </table>
      </div>
      <div id="empty" class="empty">No patients are waiting.</div>
    </div>
  </section>
</div>

<script>
let patients = JSON.parse(localStorage.getItem("emergencyPatients")) || [];
let treated = Number(localStorage.getItem("treatedCount")) || 0;
let arrivalCounter = Number(localStorage.getItem("arrivalCounter")) || 0;

function save(){
  localStorage.setItem("emergencyPatients", JSON.stringify(patients));
  localStorage.setItem("treatedCount", treated);
  localStorage.setItem("arrivalCounter", arrivalCounter);
}

```

```

function severityText(s){
    return ["", "Normal", "Mild", "Moderate", "Serious", "Critical"][s];
}

function render(){
    sortQueue();
    const body=document.getElementById("queueBody");
    body.innerHTML="";
    document.getElementById("empty").style.display=patients.length?"none":"block";

    patients.forEach((p,i)=>{
        const tr=document.createElement("tr");
        tr.innerHTML="
            <td><b>#{i+1}</b></td>
            <td>${escapeHtml(p.id)}</td>
            <td>${escapeHtml(p.name)}</td>
            <td><span class='badge s${p.severity}'>${p.severity} - ${severityText(p.severity)}</span></td>
            <td>${p.arrival}</td>
            <td>${escapeHtml(p.doctor)}</td>";
        body.appendChild(tr);
    });

    document.getElementById("total").textContent=patients.length+treated;
    document.getElementById("critical").textContent=patients.filter(p=>p.severity==5).length;
    document.getElementById("waiting").textContent=patients.length;
    document.getElementById("treated").textContent=treated;
    save();
}

document.getElementById("patientForm").addEventListener("submit",function(e){
    e.preventDefault();
    const id=document.getElementById("patientId").value.trim();
    const name=document.getElementById("patientName").value.trim();
    const severity=Number(document.getElementById("severity").value);
    const doctor=document.getElementById("doctor").value.trim();

    if(patients.some(p=>p.id.toLowerCase()===id.toLowerCase())){
        alert("Patient ID already exists.");
        return;
    }

```

5.3 Test Cases

Test Case	Input / Condition	Expected Result
TC1	Add patient with severity 5	Patient gets highest priority
TC2	Add patients with different severity	Highest severity treated first
TC3	Two patients have same severity	Earlier patient treated first
TC4	Empty queue	Display no patients waiting
TC5	Search existing patient	Patient details displayed
TC6	Search invalid Patient ID	Patient not found
TC7	Treat patient	Highest-priority patient removed
TC8	Add many patients	Heap maintains priority efficiently
TC9	Equal severity with different arrival order	FIFO maintained
TC10	Update severity	Patient priority is recalculated

5.4 Validation

The system satisfies the major requirements:

Requirement	Status
Patient registration	✓
Severity-based priority	✓
FIFO for equal severity	✓
Next patient selection	✓
Patient deletion	✓
Patient search	✓
Patient information management	✓
Doctor information	✓
Priority queue implementation	✓
Complexity analysis	✓

5.5 Time Complexity Analysis

Let n be the number of patients.

Operation	Best Case	Average Case	Worst Case
Insert	$O(1)$	$O(\log n)$	$O(\log n)$
Find Next Patient	$O(1)$	$O(1)$	$O(1)$
Delete Highest Priority	$O(\log n)$	$O(\log n)$	$O(\log n)$
Search Patient	$O(1)$	$O(n)$	$O(n)$
Update Doctor	$O(1)$	$O(n)$	$O(n)$
Update Severity	$O(\log n)$ after locating	$O(n + \log n)$	$O(n + \log n)$
Display	$O(n)$	$O(n)$	$O(n)$

Explanation

Insertion:

A new patient is initially placed at the end of the heap. It may move upward to maintain heap order. Therefore, the worst-case complexity is:

$$O(\log n)$$

Finding next patient:

The highest-priority patient is stored at the root.

$$O(1)$$

Deletion:

After removing the root, heapify-down may be required.

$$O(\log n)$$

Search:

A heap is not optimized for searching by Patient ID, so a linear search may be required.

$$O(n)$$

The use of a heap for priority processing is consistent with the general priority-queue design principle emphasized in the assignment's focus on efficient queue operations.

5.6 Space Complexity

The priority queue stores all waiting patients.

For n patients:

$$Space = O(n)$$

Additional variables used by heap operations require only constant or logarithmic auxiliary space depending on implementation.

Therefore, the overall space complexity is:

$$\boxed{O(n)}$$

6. Analysis and Engineering Decision

6.1 Conventional FIFO Queue Problem

A conventional FIFO queue processes patients strictly according to arrival time.

For example:

Patient	Arrival	Severity
P101	1	1
P102	2	5
P103	3	2

A FIFO queue would process:

P101 → P102 → P103

This is dangerous because P102 is critically ill but must wait behind P101.

A priority queue processes:

P102 → P101 → P103

Therefore, a conventional FIFO queue is not suitable for emergency patient management.

6.2 Problems with FIFO Queue

Using only a FIFO queue can result in:

1. Critical patients waiting behind minor cases.
2. Delayed treatment for life-threatening conditions.
3. Poor emergency response.
4. Increased risk to patient safety.
5. Inefficient utilization of doctors.
6. Failure to reflect medical urgency.
7. Reduced reliability of the emergency department.

The assignment specifically identifies the risk that a simple FIFO queue may cause critically ill patients to wait behind less severe patients, potentially leading to delayed treatment and serious consequences.

6.3 Comparison of Alternatives

Feature	FIFO Queue	Linked List	Array	Priority Queue
FIFO	Excellent	Good	Good	Good for equal priority
Severity priority	Poor	Moderate	Moderate	Excellent
Insert	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$
Find highest priority	$O(1)$ but wrong priority	$O(n)$	$O(n)$	$O(1)$
Delete highest priority	$O(1)$ but wrong priority	$O(n)$	$O(n)$	$O(\log n)$
Search	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Scalability	Medium	Good	Medium	Excellent
Suitability	Low	Medium	Medium	High

6.4 Final Engineering Decision

The **Priority Queue using a Binary Heap** is selected as the final solution.

The main reasons are:

- It directly supports severity-based treatment.
- The highest-priority patient can be identified in $O(1)$.
- New patients can be inserted in $O(\log n)$.
- Treated patients can be removed in $O(\log n)$.

- It scales better than repeatedly scanning an array or linked list.
- FIFO can be maintained among patients having equal severity by storing arrival order.

Therefore:

Priority Queue + Heap is the most suitable data structure

for the emergency patient management system.

7. Broader Considerations

7.1 Safety

The system gives higher priority to critical patients, helping reduce unnecessary delays in emergency treatment.

7.2 Society

Efficient emergency management can improve access to timely medical attention and support better hospital services.

7.3 Equality and Fairness

Patients with the same severity are treated according to their arrival order. This prevents arbitrary ordering among patients with equal priority.

7.4 Reliability

A structured priority queue provides predictable treatment ordering and reduces errors caused by manually managing patient queues.

7.5 Efficiency

Efficient insertion and deletion allow the system to handle a large number of patients.

7.6 Ethics

Patient information such as ID, medical severity and doctor assignment should be protected. Access should be limited to authorized hospital staff.

7.7 Professional Responsibility

A real-world hospital system is safety-critical. The software should therefore undergo extensive testing and should support human medical decision-making rather than replacing professional medical judgment.

8. Conclusion

The **Emergency Patient Management System** demonstrates how Data Structures can be applied to a real-world hospital emergency department.

A simple FIFO queue is not suitable because it treats patients according to arrival time without considering the seriousness of their conditions. This can cause critically ill patients to wait behind patients with less severe conditions.

A **Priority Queue implemented using a Binary Heap** provides a more appropriate solution.

Patients with higher severity are selected first, while patients with equal severity are processed according to their arrival order.

The system supports important operations such as patient insertion, priority assignment, next-patient selection, deletion after treatment, searching, updating, and displaying patient information.

The major complexities are:

- Insert: **$O(\log n)$**
- Find highest priority: **$O(1)$**
- Delete: **$O(\log n)$**
- Search: **$O(n)$**
- Space: **$O(n)$**

The proposed solution improves **efficiency, fairness, responsiveness and reliability** in a busy emergency department. It can be further improved by integrating real-time hospital databases, doctor availability, automated triage information, and secure patient records.

9. Student Reflection

- Through this assignment, I learned how data structures can be applied to solve practical problems in the healthcare domain. I understood that selecting a data structure depends on the actual requirements of the application.
- Initially, a simple queue may appear suitable because patients arrive continuously. However, the analysis showed that emergency departments require priority-based processing because patients have different severity levels.
- I learned how a Priority Queue and Heap can efficiently select the most important patient while maintaining FIFO order for patients with the same severity.
- I also learned about the time and space complexity of insertion, deletion, searching and updating operations. Implementing the system helped me understand the practical difference between a normal queue and a priority queue.
- If more time and resources were available, I would improve the system by adding a graphical user interface, database connectivity, automatic doctor assignment, patient status tracking, and secure authentication.

10. References

- Course material – **CSA03 Data Structures**.
- Data Structures concepts related to Queue and Priority Queue.
- Data Structures concepts related to Binary Heap.
- C Programming Language documentation.
- Assignment Question Paper – **Design and Implementation of an Efficient Priority-Based**

Emergency Patient Management System.

- Sample Data Structures Assignment Format – **Smart-City Emergency Routing System.**

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

H. Faculty Evaluation & Continuous Improvement CO

Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well:

Areas requiring improvement:

Corrective / Improvement Action

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action

 17%

Total input words: 2762

Total Matched Words from all results: 0

<

1

>

2 matches

www.bioinformatics.ccr.cancer.gov

11% similar

[bioinformatics.ccr.cancer.gov](#)
[ov > docs > unix-commands-for-beginners > unix_for_beginners](#)

Words matched: 0

Percentage matched: 11

View full comparison [Here](#)